ENDTERM PROJECT

Course: Mining Massive Data Sets

Duration: 06 weeks

I. Formation

- The project is conducted in groups of students.
- Student groups conduct designated tasks and submit the project by the given deadline.

II. Requirements

1) Task 1 (3.0 point(s)): Hierarchical clustering in non-Euclidean spaces

Implement the task in **Task01.ipynb, distributed computation** is required in this task.

Data: (1.0 point(s))

- *(0.25 point(s))* Generate a data set of about 10000 alphabetical strings whose lengths are picked randomly in the range [32, 64]. Cases are not sensitive.
- *(0.25 point(s))* Apply the 4-shingle technique to tokenize each string into a set.
- *(0.25 point(s))* Jaccard distance is used to measure shingle sets' dissimilarity.
- *(0.25 point(s))* Save the generated data set in a csv file with columns: *index*, *string*, *shingles*.

Algorithms: (1.5 point(s))

- Implement the agglomerative algorithm (bottom-up) to cluster strings.
  - Algorithm structure: agglomerative clustering
  - Data space: Euclidean
  - Memory usage: **in-memory** algorithm
  - Propose a distance threshold *t* such that clusters can be merged if and only if their distance does not exceed *t*.
- *(1.0 point(s))* Cluster representation and cluster distance approaches follow **Approach 2 of Hierarchical clustering in non-Euclidean space** presented in the corresponding lecture.
- *(0.25 point(s))* The algorithm terminates when a cluster diameter suddenly jumps. Students propose an approach to detect this abnormal event.

- *(0.25 point(s))* Organize the program regarding to the OOP model, ensure source code is compact and reasonable.

Experiments: (0.5 point(s))

- *(0.25 point(s))* Execute the algorithm throughout iterative steps.
    - For each cluster, compute the average distance between pairs of points, **avg_dist** value.
    - Compute the average of all **avg_dist** values of clusters, named **global_avg_dist**.
    - Draw a line chart to illustrate the improvement of **global_avg_dist** throughout steps.
- *(0.25 point(s))* Use **t-Distributed** Stochastic **Neighbor Embedding** (t-SNE) algorithm to visualize the clustering result in a 3D space.

2) Task 2 (3.0 point(s)): Linear Regression – Gold price prediction

Implement the task in **Task02.ipynb**, **distributed computation** is required in this task.

Data: (0.5 point(s))

- Given a data set of gold prices in Vietnam, collected from 2009/08/01 to 2025/01/01, in the **gold_prices.csv** file.
- Columns include [*Date, Buy Price, Sell Price*].
- Use PySpark to read the given data set and transform it into a data frame.
- *(0.25 point(s))* Generate samples with features and labels as below
    - Features: gold prices of 15 consecutive previous dates of the date *t*,
    - Label: the gold price of the date *t*.
- *(0.25 point(s))* Divide samples randomly into two sets, including training and test, with the ratio 7:3.

Algorithms: (1.5 point(s))

- *(0.5 point(s))* Students are allowed to use the existing Linear Regression model of PySpark instead of manually implementing.

`pyspark.ml.regression.LinearRegression`

- *(1.0 point(s))* Use PySpark to implement the CUR algorithm, in form of a class, to reduce dimensionality of feature vectors from 15 to 5 with the step size of 1.

- Students are allowed to use the **RowMatrix** library of PySpark

```
pyspark.mllib.linalg.distributed.RowMatrix
```

Experiments: (1.0 point(s))

- *(0.25 point(s))* Infer the new representation (row embedding) for each feature vector in the training and test sets.
- Train and evaluate linear regression models in the pairs of training and test sets.
  - *(0.25 point(s))* Draw a line chart to illustrate training losses in experiments (a single chart only with multiple loss curves).
  - *(0.25 point(s))* Draw a twin-bar chart to contrast the results in the training and test sets in experiments (a single chart only with multiple twin-bars).
- *(0.25 point(s))* Experiments are implemented in form of classes with configurations as parameters instead of replicating blocks of code.

3) Task 3 (3.0 point(s)): Collaborative Filtering for Recommendation

Implement the task in **Task03.ipynb**, **distributed computation** is required in this task.

Data: ***(0.5 point(s))***

- Given a dataset of ratings in **ratings2k.csv**.
- *(0.25 point(s))* Each user profile is represented as a vector ***u*** of ratings of items.
- u[it] = 0 means the user ***u*** has not tried/voted the item ***it*** yet. It does not mean ***u*** does not like ***it*** (negative).
- *(0.25 point(s))* Propose a distance metric to measure the similarity between two users, in which ensure that 0s do not mean 'negative'.

Algorithms: ***(1.5 point(s))***

- Implement the Collaborative Filtering algorithm to predict the rating of a user ***u*** for an item ***it***.
  - *(0.5 point(s))* Propose an approach to two "quickly" determine users that are most similar (close) to ***u*** without enumerating the whole data set. The time complexity of O(1) in average is expected.
  - *(0.5 point(s))* Implement the proposed approach and integrate it into the processing pipeline of the Collaborative Filtering algorithm.
  - *(0.25 point(s))* Compute the predicted rating based on user-user similarity.

- *(0.25 point(s))* Organize the program regarding to the OOP model, ensure source code is compact and reasonable.

Experiments: *(1.0 point(s))*

- *(0.25 point(s))* Divide the given data set into training and test sets.
- *(0.25 point(s))* Conduct experiments with various values of N.
- *(0.25 point(s))* Draw a bar chart to contrast RMSE values resulted by selected values of N.
- *(0.25 point(s))* Compare the running time (in the same hardware configurations) when running the algorithm with versus without the proposed approach of determining similar users.

4) Task 4 (1.0 point(s)): Report

- Student groups compose the project report using the IEEE conference proceeding template.
- Recommended editor: Overleaf.
- Selective contents:
  - *Title*: the project title
  - *Authors*: group member's information, the lecturer is appended as the last author.
  - *Abstract*: summarize the project requirements, approaches, experimental results, and levels of completion.
  - Each following section presents a task in the project, with a meaningful and human-readable title. Briefly introduce the approach to tackle the problem and illustrate results with related figures/tables, etc.
  - "*Contributions*" section: individual tasks, individual completion levels (0%-100%).
  - "*Self-evaluation*" section: self-evaluate task completion and estimate scores.
  - "*Conclusion*" section: summarize the project requirements, approaches, experimental results, and levels of completion.
- References are in the IEEE format.
- Maximal length is **05 pages**.

III. Submission Notice

- Create a folder whose name is like

final_<Project group ID>_<your student ID>

  - **Source/:** consists of the project source code, each task is implemented in an individual sub-directory, preserving the outputs of all cells in ipynb files, output files as well.
  - **Report/:** report source (exported from Overleaf), **report.pdf** file.
- Compress the folder as a zip file and submit by the deadline.
- Every team member must submit the project individually.

IV. Policy

- **Student groups submitting late get 0.0 points for each member.**
- **Copying source code on the internet/other students, sharing your work with other groups, etc., cause 0.0 points for all related groups.**
- **If there exist any signs of illegal copying or sharing of the assignment, then extra interviews are conducted to verify student groups' work.**

-- THE END --